# plmc

A minimal PL/M compiler for the Motorola 6809 – user manual

plm09 project

October 2026

## Contents


**1 Introduction** **2**

**2 Building and quick start** **3**

**3 Command-line usage** **3**

**4 The language** **4**

4.1 Lexical rules . . . . . 4

4.2 Program structure . . . . . 5

4.3 Declarations . . . . . 5

4.4 LITERALLY . . . . . 6

4.5 Arrays . . . . . 6

4.6 INITIAL and DATA . . . . . 7

4.7 Structures . . . . . 8

4.8 BASED variables and the dot operator . . . . . 9

4.9 AT . . . . . 10

4.10 Procedures . . . . . 11

4.11 Statements . . . . . 13

4.12 Labels and GOTO . . . . . 14

4.13 Iterative DO . . . . . 15

4.14 DO CASE . . . . . 16

4.15 Expressions . . . . . 16

4.16 Types and conversions . . . . . 17

4.17 Conditions . . . . . 18

4.18 Unsupported features . . . . . 18

**5 Diagnostics** **18**

**6 Hooks** **21**

6.1 How targets are selected . . . 21

6.2 What a hooks file must define . . . 22

6.3 Calling convention for EXTERNAL procedures . . . 22

6.4 Hooks provided . . . 23

6.5 Writing hooks for a new machine . . . 23

**7 OS-9 modules** **23**

7.1 What `--os9` changes . . . 24

7.2 Module names . . . 25

7.3 The OS-9 hooks . . . 25

**8 Running on a Color Computer** **26**

**9 Generated code** **26**

9.1 Layout . . . 26

9.2 Example . . . 27

9.3 Conventions . . . 27

**10 Inspecting the compiler's view** **30**

**11 Testing** **30**

**12 Source code** **31**

# 1 Introduction

`plmc` compiles programs written in a small subset of PL/M-80, Intel's systems programming language of the late 1970s, into assembly source for the Motorola 6809. Its output is meant for **lwasm**, the assembler in the lwtools package (version 4.24 or later). lwasm then produces whatever binary format the target needs: S-records, Disk BASIC `.BIN` files, raw images and so on.

The compiler is not tied to any particular machine. Everything that depends on the hardware lives in a small **hooks file** that lwasm includes, chosen with an include path. Four hooks files come with plmc:

| Hooks | Target |
|---|---|
| `runtime/sim/` | `sim6809`, the headless simulator used by the test suite |
| `runtime/coco/` | the Tandy Color Computer 1, 2 and 3 under Color BASIC |
| `runtime/os9/` | OS-9 and NitrOS-9 (Level 1 and 2), for modules built with `--os9` |
| `runtime/example/` | a template for a board with an MC6850 ACIA serial port |

### Main features

- BYTE (8-bit) and ADDRESS (16-bit) unsigned variables, arrays and structures, with PL/M-80's rules for mixing the two types, and the built-ins `LENGTH`, `LAST` and `SIZE`.
- All PL/M arithmetic, logical and relational operators, with PL/M precedence.
- `LITERALLY` macros, scoped like other declarations.
- Pointers: `BASED` variables and the address-of (dot) operator `.X`.
- `INITIAL` and `DATA` initializations, including strings of any length up to 255 characters, and `(*)` dimensions.
- `IF`/`THEN`/`ELSE`, `DO` blocks with their own declarations, `DO WHILE` and iterative `DO I = ... TO ... BY ...` loops, `DO CASE`, labels and `GOTO`, `HALT`.
- Procedures: typed and untyped, with parameters, nested inside each other, and `EXTERNAL` procedures written in assembly.
- Error messages that give the file, line and column. Every unsupported PL/M feature is rejected with a clear message rather than misread.
- Readable output: each source statement appears as a comment above the code generated for it.
- With `--os9`, position-independent, reentrant OS-9 program modules that run under OS-9 and NitrOS-9.

# 2 Building and quick start

`plmc` is plain C11. From the top of the plm09 tree:

```
make            # builds build/plmc and build/sim6809
make test       # runs every test suite
```

A first program:

```
SUM: DO;
PUTDEC: PROCEDURE (N) EXTERNAL; DECLARE N ADDRESS; END PUTDEC;
DECLARE I BYTE, TOTAL ADDRESS;
TOTAL = 0;
I = 1;
DO WHILE I <= 10;
    TOTAL = TOTAL + I;
    I = I + 1;
END;
CALL PUTDEC(TOTAL);
END SUM;
```

Compile it, assemble it, and run it in the simulator:

```
$ build/plmc -o sum.asm sum.plm
$ lwasm -9 -f srec -I runtime -I runtime/sim -o sum.s19 sum.asm
$ build/sim6809 sum.s19
55
```

`make run F=sum.plm` does all three steps. Always pass `-9` to lwasm (or rely on the `PRAGMA 6809` that plmc emits). By default lwasm accepts the Hitachi 6309's extra instructions, which a 6809 doesn't have.

# 3 Command-line usage

```
plmc [options] file.plm
```

| Option | Meaning |
|---|---|
| `-o FILE` | Write the assembly to *FILE*. The default is standard output. |
| `--org ADDR` | Address where the program is loaded and starts. The default is `$0100`. |
| `--vectors` | Also emit `ORG $FFFE` / `FDB RT_START`, setting the 6809 reset vector, for programs that run from ROM or straight from reset, and the IRQ, FIRQ and NMI vectors of any INTERRUPT procedures. |
| `--os9` | Generate a position-independent OS-9 program module instead of a program for a fixed address. See OS-9 modules. |
| `--stack N` | With `--os9`: bytes of stack to reserve in the module's data area (default 256, at least 32). |
| `--name NAME` | With `--os9`: the module name (default: the PL/M program's name). |
| `--tokens` | Print the token stream and stop. See Inspecting the compiler's view. |
| `--ast` | Print the syntax tree and stop. |
| `-h`, `--help` | Print a usage summary and exit. |

The `--org` address and `--stack` size can be written as decimal (`8192`), `0x2000`, `$2000` (quote it for the shell) or PL/M style `2000H`, and must fit in 16 bits. `--org` and `--vectors` can't be combined with `--os9`, and `--stack` and `--name` need `--os9`; violations are reported as errors.

**Exit status**

| Status | Meaning |
|---|---|
| 0 | Success. Any warnings are still printed. |
| 1 | Compilation error, or a file couldn't be read or written. If `-o` was given and writing failed, the partial output file is removed. |
| 2 | Bad usage, for example no input file or an unknown option. |

# 4 The language

plmc accepts a **subset** of PL/M-80. Anything outside the subset is rejected with an error (see Unsupported features), so a program that compiles means what PL/M-80 says it means.

## 4.1 Lexical rules

- **Case:** case doesn't matter. `Total`, `TOTAL` and `total` are the same name, and keywords may be written in any case.
- **The `$` character:** ignored inside names and numbers, so you can use it as a separator: `MAX$COUNT`, `1$000`, `0FF$FFH`.
- **Names:** start with a letter and continue with letters, digits and `$`. The underscore is **not** allowed (as in PL/M-80). This is what keeps compiler-generated labels from ever colliding with your names.

- **Comments:** `/* ... */`. They may span several lines but don't nest.
- **Numbers:** must start with a digit and fit in 16 bits (0 to 65535). An optional suffix gives the base:

| Suffix | Base | Example | Value |
|---|---|---|---|
| none or `D` | decimal | `255`, `255D` | 255 |
| `H` | hexadecimal | `0FFH` | 255 |
| `O` or `Q` | octal | `377Q` | 255 |
| `B` | binary | `1111$1111B` | 255 |

  A hexadecimal number has to start with a digit, so write `0FFH`, not `FFH` (`FFH` is a name).

- **Strings:** a one-character string such as `'A'` is a BYTE constant (65). A two-character string such as `'AB'` is an ADDRESS constant (`'A'`× 256 + `'B'`). Write a quote character as `''`, as in `''''`. A longer string, up to 255 characters, can't be a number. As in PL/M-80 it can be used in three places:
  - an `INITIAL` or `DATA` list, as in `DECLARE MSG (*) BYTE DATA ('HELLO')`
  - a constant list, `.('HELLO', 0)`, which gives its address
  - the text of a `LITERALLY`

  Used as a value anywhere else, it is an error.

- **Reserved words:** all PL/M-80 reserved words, including ones for features plmc doesn't implement, so they can't be used as names: `ADDRESS AND AT BASED BY BYTE CALL CASE DATA DECLARE DISABLE DO ELSE ENABLE END EOF EXTERNAL GO GOTO HALT IF INITIAL INTERRUPT LABEL LITERALLY MINUS MOD NOT OR PLUS PROCEDURE PUBLIC REENTRANT RETURN STRUCTURE THEN TO WHILE XOR`.

## 4.2 Program structure

A program is one labeled block:

```
NAME: DO;
    declarations
    statements
END NAME;
```

The name after `END` is optional, but if present it must match. The block may optionally be followed by the keyword `EOF`. Within any block, **all declarations must come before the first statement.**

## 4.3 Declarations

```
DECLARE COUNT BYTE;
DECLARE (X, Y, Z) ADDRESS;
DECLARE FLAG BYTE, LIMIT ADDRESS, (LO, HI) BYTE;
```

- **Types:** `BYTE` is an unsigned 8-bit value (0 to 255) and `ADDRESS` an unsigned 16-bit value (0 to 65535).
- **Order and scope:** names must be declared before they are used. A name declared in an inner block or procedure hides the same name from an outer one. Declaring a name twice in the same block is an error.
- **Initial values:** a variable has no initial value unless it is declared with `INITIAL` or `DATA` (see INITIAL and DATA). Otherwise it starts with whatever is in memory, which is `$A5` in the simulator so that the mistake shows up.

## 4.4 LITERALLY

```
DECLARE TRUE LITERALLY '0FFH', FALSE LITERALLY '0';
DECLARE BUFFER$SIZE LITERALLY '32';
DECLARE DCL LITERALLY 'DECLARE', PROC LITERALLY 'PROCEDURE';

DCL PRINT$BUFFER(BUFFER$SIZE) BYTE;
```

`DECLARE name LITERALLY 'text'` defines a parameterless macro. Every later occurrence of *name* is replaced by the tokens of *text*, before parsing, so a literal can stand for anything:

- a constant (`TRUE`, `BUFFER$SIZE`), including where only a number is allowed, such as an array dimension
- a keyword (`DCL`, `PROC`)
- part of an expression
- a whole statement (`INCR LITERALLY 'N = N + 1'`, then `INCR;`)

The rules:

- **Text:** up to 255 characters. Write a quote inside it as `''`, so `QUOTE LITERALLY '''A'''` stands for `'A'`.
- **Nesting:** a literal's text may use other literals, which are expanded in turn. A literal that expands to itself is reported as `literal X expands recursively`.
- **Scope:** literals follow block scope like any other declaration.
  - A literal declared in a procedure or a `DO` block is visible only there, from its declaration on.
  - An inner block can declare its own literal, or a variable or procedure, with the same name, which hides the outer literal until that block's `END`.
  - Declaring the same name twice in one block is an error.
- **Names being defined are not expanded:** declared names, procedure names, parameters, labels and the name after `END`.
- **Textual substitution:** the text is substituted *as text*, so precedence can surprise you. After `DECLARE TWO LITERALLY '1 + 1'`, the expression `TWO * 3` means `1 + 1 * 3`, which is 4. Put parentheses in the literal (`'(1 + 1)'`) when it is an expression.
- **Form:** a LITERALLY can't be factored (`DECLARE (A, B) LITERALLY ...`) or have a dimension, but it can be mixed with other items in one `DECLARE`.
- **Seeing the result:** `--ast` shows the program *after* substitution, which is the easiest way to check what a literal turned into.

## 4.5 Arrays

```
DECLARE BUF(80) BYTE;
DECLARE (XS, YS)(10) ADDRESS;
DECLARE COUNT BYTE, TABLE(16) ADDRESS;
```

- **Declaring:** a dimension in parentheses makes an array. `BUF(80)` has the 80 elements `BUF(0)` to `BUF(79)`. The dimension must be a number from 1 to 65535, and the array must fit in 64 KB.
- **Element type:** the elements are BYTE or ADDRESS, and an element behaves exactly like a variable of that type in expressions and conversions.
- **Using an element:** `BUF(I)` is an element wherever a variable can appear: in expressions and as an assignment target, including multiple assignment (`BUF(I), BUF(J), N = 0;`).

- **Subscripts:** any expression, BYTE or ADDRESS. A BYTE subscript is zero-extended. **Subscripts are not checked:** as in PL/M-80, `BUF(80)` silently reads or writes the memory after the array.
- **Unsubscripted name:** a plain `BUF` means `BUF(0)`.
- **Not allowed:** arrays can't be parameters, and procedures can't return them. Pass an element (its value), or keep the array global, or in an outer procedure that the procedure can see.
- **Built-ins:** three PL/M-80 functions describe a variable, as constants worked out by the compiler:
  - `LENGTH(A)`: the number of elements
  - `LAST(A)`: the highest subscript, `LENGTH(A) - 1`
  - `SIZE(X)`: the number of bytes

  `LENGTH` and `LAST` are **BYTE** when the value is 255 or less and ADDRESS otherwise, as in PL/M-80. Arithmetic with them follows the BYTE rules: if `LENGTH(B)` is 10, then `LENGTH(B) + 250` wraps to 4. `SIZE` is always an ADDRESS.

  For a scalar, `LENGTH` and `LAST` give 1 and 0. The built-ins are only used when you haven't declared a procedure or variable with the same name yourself. They also accept structure references; see Structures.

```
I = 0;
DO WHILE I < LENGTH(BUF);
    BUF(I) = ' ';
    I = I + 1;
END;
```

## 4.6 INITIAL and DATA

```
DECLARE THRESHOLD BYTE INITIAL (48);
DECLARE (COUNTER, LIMIT, INCR) ADDRESS INITIAL (0, 1024, 2);
DECLARE GREETING (*) BYTE INITIAL ('HELLO, WORLD', 0);
DECLARE DAYS (*) BYTE DATA (31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31);
DECLARE MESSAGES (*) ADDRESS INITIAL (.GREETING, .('BYE', 0));
```

An `INITIAL (...)` or `DATA (...)` list after the type gives a declaration initial values (PL/M-80 manual 6.2.9).

**Values.** The list fills the declaration's **scalars in storage order**:

- array elements from 0 up
- a structure's members in order, including member arrays, and each structure of an array of structures in turn
- across all the names of a factored declaration: `INITIAL (0, 1024, 2)` above sets COUNTER, LIMIT and INCR

Each value is one of these:

| Value | Fills |
|---|---|
| a number, as in `48`, `0FFH` or `'A'` | one scalar; a BYTE needs 0 to 255 |
| a string, `'HELLO'` | one character per BYTE, two per ADDRESS (`'ABC'` fills two ADDRESS scalars with `'AB'` and `'C'`) |

| Value | Fills |
|---|---|
| a location reference plus or minus numbers, as in `.BUF`, `.TABLE(2) - 2`, `.('TEXT', 0)` or `.PROC` | one ADDRESS scalar with that address. Subscripts must be constants, the variable must be declared earlier and not BASED |

**Length of the list.** The list may be shorter than the declaration, and the remaining scalars are then left uninitialized. It may not be longer.

**Implicit dimensions.** `(*)` in place of a dimension makes the array exactly as long as its list. `FAREWELL (*) BYTE INITIAL ('GOODBYE, NOW')` has 12 elements, and `LENGTH(FAREWELL)` is 12. `(*)` can't be used in a factored declaration, for an array of structures, or for a member.

**INITIAL or DATA:**

- `INITIAL` gives a variable its value when the program is **loaded**. The program can change it afterwards; it isn't reset on a restart. `INITIAL` is allowed only at the **outer level of the main program**.
- `DATA` stores the values **with the program code**, so they could be in ROM. A DATA variable is a constant table, and the compiler rejects assignments to it. `DATA` is allowed at any level, including inside procedures.
- **Not allowed:** a BASED variable or a parameter can't be initialized.

## 4.7 Structures

```
DECLARE AIRPLANE (20) STRUCTURE (SPEED BYTE, ALTITUDE ADDRESS);
DECLARE PAYROLL (100) STRUCTURE (
    LAST$NAME (15) BYTE,
    FIRST$NAME (15) BYTE,
    DOLLARS ADDRESS);

AIRPLANE(17).ALTITUDE = 9000;
PAYROLL(N).FIRST$NAME(K) = 'A';
```

A **structure** groups members of different types under one name. `STRUCTURE (member, ...)` is a type, written where `BYTE` or `ADDRESS` would be, so it can be combined with the other parts of a declaration:

- **Forms:**
  - a single structure: `DECLARE POINT STRUCTURE (X BYTE, Y BYTE);`
  - an array of structures: `DECLARE AIRPLANE (20) STRUCTURE (...)`
  - factored: `DECLARE (A, B) STRUCTURE (...)`, which gives both the same members
  - BASED: `DECLARE ITEM BASED P STRUCTURE (...)`
- **Members:** each is `name [(dimension)] BYTE` or `ADDRESS`.
  - A member can be an array.
  - A member can't be factored, BASED, or itself a structure; structures don't nest.
  - Member names belong to their structure, so different structures can use the same names, and a member name never clashes with a variable.
- **Layout:** the members are stored one after another in the order written, so `SIZE` of a structure is the sum of its members' sizes. An array of structures stores whole structures one after another.

**References.** A member is referenced as `X.M`, `X(i).M`, `X.M(j)` or `X(i).M(j)`. Each subscript can be any expression, and each is **unchecked**, as for arrays. A member reference can be used anywhere a variable can:

in expressions, as an assignment target, and with `..` An unsubscripted member array (`POINT.COORD`) means its element 0.

A whole structure (`POINT`, or `AIRPLANE(3)`) is not a value. It can only be used in these places:

| Reference | Meaning |
|---|---|
| `.POINT`, `.AIRPLANE(3)`, `.AIRPLANE` | the address of the structure (of element 0 for an array) |
| `.PAYROLL(5).FIRST$NAME` | the address of a member array |
| `LENGTH(AIRPLANE)`, `LAST(AIRPLANE)` | the number of structures in an array of structures, and the last subscript |
| `SIZE(POINT)`, `SIZE(AIRPLANE)`, `SIZE(AIRPLANE(3))` | bytes in a structure, the whole array, or one element |
| `LENGTH(PAYROLL(5).FIRST$NAME)`, `SIZE(...)` | a member array's length and size |

**The built-in shorthand.** A member of an *array* of structures normally needs the array's subscript (`AIRPLANE.SPEED` is an error). The built-ins are the exception, because every element's member array has the same size: `LENGTH(PAYROLL.FIRST$NAME)` is 15.

**Pointers.** Structures combine naturally with BASED variables and the dot operator. For example, a procedure can take a structure's address and look at it through a BASED structure:

```
TOTAL: PROCEDURE (Q) ADDRESS;
    DECLARE Q ADDRESS, R BASED Q STRUCTURE (KEY BYTE, INFO (3) ADDRESS);
    ...  R.INFO(K) ...
END TOTAL;

X = TOTAL(.RECORD);
```

A linked list can be built in an array of structures with an ADDRESS member holding `.NODE(i)`, and walked with a BASED structure.

**Not allowed:** structures can't be parameters, values, or iterative DO indexes. Pass `.X` and use a BASED structure instead.

## 4.8 BASED variables and the dot operator

```
DECLARE (NORTH$ERROR, EAST$ERROR) BYTE;
DECLARE ERROR$POINTER ADDRESS;
DECLARE ERROR BASED ERROR$POINTER BYTE;

ERROR$POINTER = .NORTH$ERROR;
ERROR = 0;                          /* clears NORTH$ERROR */
```

A **BASED variable** has no storage of its own. Each reference to it uses the memory whose address is, *at that moment*, in its **base**.

- **Declaring:** `BASED base` must come right after the variable's name:
  - `DECLARE ITEM BASED PTR BYTE;`
  - `DECLARE (A BASED P, B BASED Q) ADDRESS;`
  - `DECLARE (LIST BASED LP) (100) ADDRESS;` (the parentheses are optional)

- **The base:** an ADDRESS scalar declared earlier. It can't be an array, an element, or itself BASED. Parameters and locals make good bases.
- **Arrays:** a BASED array's element `A(i)` is at `base + i` (BYTE) or `base + 2*i` (ADDRESS), and `A` alone means `A(0)`. As with ordinary arrays nothing is range-checked, so `DECLARE (BUF BASED P) (1) BYTE` can index any amount of memory.
- **Not allowed:** a BASED variable can't be a parameter or the index of an iterative DO.

The **dot operator** gives an ADDRESS value:

| Form | Value |
|---|---|
| `.X` | the address of variable `X` (of element 0 if `X` is an array) |
| `.A(i)` | the address of element `A(i)` |
| `.B` for a BASED `B` | the current value of its base (plus the offset for `.B(i)`) |
| `.P` | the entry address of procedure `P`. `P` must be declared at the outer level, and is written without arguments |
| `.(constants)` | the address of the listed bytes, stored with the program: numbers from 0 to 255 and strings of any length up to 255, as in `.('HELLO', 13, 0)` |

Together they give PL/M its pointers: pass `.BUF` to a procedure whose parameter is the base of a BASED array, walk a string by adding 1 to a base, or keep tables of addresses. The 6809 stores ADDRESS values high byte first, so a BYTE BASED on an ADDRESS variable sees its high byte.

In an `--os9` module, `.X` gives the variable's actual run-time address inside the process's data area, and constant lists live in the module.

## 4.9 AT

```
DECLARE BUFFER (3) BYTE;
DECLARE (CHAR$A, CHAR$B, CHAR$C) BYTE AT (.BUFFER);
DECLARE STATUS BYTE AT (0FF00H);
```

`AT (address)` places a variable at an address of your choosing instead of giving it storage of its own (PL/M-80 manual 6.2.8).

- **Placement:** the first name of the declaration is at the address, and any others follow it in order, so `CHAR$A`, `CHAR$B` and `CHAR$C` above are `BUFFER(0)`, `BUFFER(1)` and `BUFFER(2)`. `AT` goes after the type and before any `INITIAL`.
- **The address** is a restricted expression, as for `INITIAL`:
  - a number, as in `AT (0FF00H)`: the variable is at that fixed address, for example a memory-mapped I/O register. Plus or minus numbers is allowed.
  - the location of a variable declared earlier, plus or minus numbers: `AT (.BUFFER)`, `AT (.TABLE(3) - 2)`, `AT (.REC.VAL(1) + 1)`. The new variable is an **alias**: it shares that variable's storage, wherever that is. Subscripts must be constants.
- **Byte order:** the 6809 is big-endian, so `DECLARE ITEM BYTE AT (.DATUM)`, where `DATUM` is an ADDRESS, is the *high-order* byte of `DATUM` (on the 8080 it would be the low-order byte). Use `AT (.DATUM + 1)` for the low-order byte.
- **Any type:** scalars, arrays and structures can be declared `AT`, in procedures as well as in the main program.

- **With INITIAL:** the values are stored at the address, as part of the load image (`ORG` to the address, then `FCB`/`FDB`). In an `--os9` module they are copied there at start-up.
- **Not allowed:** `AT` with `BASED`, with `DATA`, for a parameter, or with the address of a procedure, of a constant list or of a BASED variable.
- **In `--os9` modules:** an alias stays in the data area (`,U`) or, if it is AT a DATA variable, in the module (`,PCR`). A variable AT a number is used at that absolute address, which is fine for I/O registers but leaves the program's own memory.

## 4.10 Procedures

```
NAME: PROCEDURE [(PARAM, ...)] [BYTE | ADDRESS] [EXTERNAL | REENTRANT]
                [INTERRUPT n];
    declarations
    statements
END [NAME];
```

- **Parameters:** each one must be declared, with its type, in the procedure's own declarations.
- **Typed and untyped:**
  - A typed procedure (`BYTE` or `ADDRESS`) returns a value with `RETURN expression;` and is used inside expressions.
  - An untyped procedure is invoked with `CALL` and may use a bare `RETURN;`.
  - Using one kind where the other is expected is an error.
- **No arguments:** a typed procedure with no parameters is called just by writing its name: `X = NEXTCHAR;`.
- **Nesting:** procedures may be nested. An inner procedure can use the parameters and variables of the procedures around it, and of the program.
- **Argument conversion:** arguments are converted to the parameter's declared type, and `RETURN` values to the procedure's type. See Types and conversions.
- **Reaching `END`:** an untyped procedure just returns. A typed procedure that might reach its `END` without a `RETURN` gets a warning, and the value it returns is then undefined.
- **Static storage:** as in classic PL/M-80, the parameters and local variables of an ordinary procedure live at fixed memory locations, not on the stack. Such a procedure must not call itself, directly or through other procedures, unless it is `REENTRANT`. plmc reports both cases:

  ```
  error: procedure A is recursive (A -> B -> A) but not REENTRANT
  ```

### 4.10.1 REENTRANT procedures

```
SUM$ARRAY: PROCEDURE (PTR, N) BYTE REENTRANT;
    DECLARE PTR ADDRESS,
        ARRAY BASED PTR (1) BYTE,
        N BYTE;
    IF N=0 THEN RETURN ARRAY(0);
    ELSE RETURN ARRAY(N) + SUM$ARRAY(PTR, N-1);
END SUM$ARRAY;
```

A REENTRANT procedure keeps its parameters and variables on the stack, so every activation has its own (PL/M-80 manual 8.1.7). It may call itself, directly or through other procedures, and it may be called from an interrupt procedure while it is running.

- **Calling ahead:** inside a REENTRANT procedure, a REENTRANT procedure may be called (or have its address taken) before it is declared. That allows indirect recursion: IS$EVEN can call IS$ODD, declared after it, which calls IS$EVEN. Anywhere else, names must be declared first.
- **Cycles:** every procedure on a cycle of calls must be REENTRANT.
- **What is on the stack:** the parameters, and the variables declared in the procedure and in its DO blocks, including arrays and structures. Exceptions: DATA (read-only, kept with the code), and variables AT a number or AT a variable that is not on the stack.
- **Rules:**
  - only at the outer level of the main program;
  - no procedure declarations inside it;
  - not EXTERNAL;
  - its variables take at most 32767 bytes.
- **Stack use:** each activation needs its variables, its arguments and 4 bytes, plus room for intermediate results. Deep recursion needs a big enough stack; with --os9, raise --stack (default 256 bytes).
- **Pointers:** .X of a variable on the stack is its address in the current activation, valid until the procedure returns.

### 4.10.2 INTERRUPT procedures, ENABLE and DISABLE

```
DECLARE PIA0B BYTE AT (0FF02H);
TICK: PROCEDURE INTERRUPT 0;
    DECLARE ACK BYTE;
    ACK = PIA0B;                    /* acknowledge the device */
    TICKS = TICKS + 1;
END TICK;
```

INTERRUPT n makes a procedure run when interrupt $n$ occurs (PL/M-80 manual 8.1.6). The 8080's numbers are RST levels; on the 6809 they are:

| n | Interrupt | Vector | Registers stacked by the CPU |
|---|---|---|---|
| 0 | IRQ | $FFF8 | all |
| 1 | FIRQ | $FFF6 | PC and CC (plmc's entry code saves the others) |
| 2 | NMI | $FFFC | all |

- **What happens:** the CPU masks interrupts (IRQ sets I; FIRQ and NMI set I and F) and enters the procedure through a small entry routine, RT_INTn, which sets DP = 0 and calls it. When the procedure returns (RETURN or END), RTI restores the interrupted program's registers, including CC, so interrupts are enabled again.
- **Acknowledge the device:** IRQ and FIRQ stay asserted until the device that raised them is serviced. The procedure must do that (typically by reading or writing a device register, declared AT its address), or it will be entered again as soon as it returns.
- **Installation:** at start-up the program calls the hooks' PLM_SETINT for each INTERRUPT procedure, which points that interrupt at RT_INTn. The simulator's hooks store the vector; the CoCo's patch BASIC's RAM vector (and PLM_EXIT puts it back). For a ROM-based system, --vectors also writes RT_INTn into the vector table.
- **ENABLE; and DISABLE;** clear and set both the I and F masks (ANDCC #$AF, ORCC #$50). NMI can't be disabled. Programs start with interrupts as the hooks leave them: masked after a reset (and in the simulator), enabled under Color BASIC.

- **CALL:** an interrupt procedure may also be called like any untyped procedure. Interrupts aren't masked for such a call, and when it returns they are enabled, as by `ENABLE`.
- **GOTO out:** an interrupt procedure may `GOTO` a label at the outer level of the main program. Control never returns to the interrupted code, and interrupts stay masked until `ENABLE`.
- **HALT** enables interrupts before it stops the program, as in PL/M-80.
- **Shared procedures:** a procedure that might be running when an interrupt procedure calls it again must be `REENTRANT` (its static variables would otherwise be overwritten). The runtime routines (`RT_MUL16`, `RT_DIV16`) use only the stack and are safe.
- **Rules:**
  - untyped, with no parameters;
  - only at the outer level of the main program;
  - not `EXTERNAL`;
  - each number at most once, and only 0, 1 and 2;
  - not in `--os9` modules (OS-9 owns the interrupts), and neither are `ENABLE` and `DISABLE`.

### 4.10.3 EXTERNAL procedures

An `EXTERNAL` procedure is written in assembly language, usually in the hooks file. Its PL/M declaration gives only the parameters and their types:

```
PUTCH: PROCEDURE (C) EXTERNAL;
    DECLARE C BYTE;
END PUTCH;
```

The body may contain only the parameter declarations. In the assembly the procedure is called by its exact PL/M name, so it must not be a 6809 register name (`A`, `B`, `D`, `X`, `Y`, `U`, `S`, `PC`, `CC`, `DP`, `PCR`). The calling convention is described under Hooks.

## 4.11 Statements

| Statement | Form |
|---|---|
| Assignment | `X = expression;` or `A(i) = expression;` |
| Multiple assignment | `X, A(i), Z = expression;` (evaluated once, converted for each target) |
| Call | `CALL NAME;` or `CALL NAME(arg, ...);` (untyped procedures only) |
| Conditional | `IF condition THEN statement; [ELSE statement;]` |
| Block | `[LABEL:] DO; declarations statements END [LABEL];` |
| Loop | `[LABEL:] DO WHILE condition; statements END [LABEL];` |
| Counted loop | `[LABEL:] DO I = start TO limit [BY step]; statements END [LABEL];` (see Iterative DO) |
| Case | `[LABEL:] DO CASE expression; statement-0; statement-1; ... END [LABEL];` (see DO CASE) |
| Return | `RETURN;` or `RETURN expression;` (inside procedures only) |

| Statement | Form |
|---|---|
| Halt | `HALT;` stops the program through the hooks' `PLM_HALT` (enabling interrupts first if the program has INTERRUPT procedures) |
| Interrupt masks | `ENABLE;` and `DISABLE;` (see INTERRUPT procedures, ENABLE and DISABLE) |
| Empty | `;` |
| Jump | `GOTO label;` or `GO TO label;` (see Labels and GOTO) |

- **ELSE:** an `ELSE` belongs to the nearest `IF` that doesn't already have one. To put several statements under `THEN` or `ELSE`, wrap them in `DO; ... END;`.
- **Labels:** any executable statement may carry labels, `L1: L2: stmt. END name` after a `DO` block must match one of the DO's labels. See Labels and GOTO.
- **Loop and case blocks:** `DO WHILE`, iterative `DO` and `DO CASE` blocks may not contain declarations at their top level. A simple `DO` block inside them may.

## 4.12 Labels and GOTO

```
        I = 0;
AGAIN:  I = I + 1;
        IF I < 10 THEN GOTO AGAIN;

        DO WHILE 1;
            IF DONE THEN GOTO FINISHED;     /* leave the loop */
            ...
        END;
FINISHED: ;
```

**Labels** (PL/M-80 manual 5.3, 9.3):

- **Where:** a label can be put on any executable statement, and a statement can have several (`L1: L2: N = 0;`). A labeled null statement `L: ;` marks a place, for example just before an `END`. Declarations and procedures can't be labeled.
- **Scope:** a label belongs to the **smallest block** that contains it. Every `DO` block counts as a block (simple, WHILE, iterative and CASE), and so does every procedure, but an `IF` does not.
  - A label may be used **before** it is defined.
  - The same name can be a label in two different blocks.
  - A label shares the block's names with its variables and procedures, so a clash is an error.
- **Labels on a DO:** a label on a `DO` statement belongs to the enclosing block, not to the DO's own block. So `GOTO L` to the label of `L: DO; ... END;` runs the whole block again.
- **The program's name:** the label on the program's first `DO` is outside every block, and can't be a GOTO target.

**GOTO** transfers control to a label that is visible from where the GOTO is:

- **Allowed:** a label in the same block, or in any block that encloses it. This lets you leave loops, `DO CASE` blocks and nested blocks early.
- **Not allowed:** jumping **into** a block. Its labels aren't visible from outside, so this is reported as `L is not declared`.

- **Leaving a procedure:** a GOTO in a procedure may jump out of it, even through several nested calls, but **only to a label at the outer level of the main program**. Anything else is an error. plmc then discards everything those calls left on the stack, which is safe because procedures keep their parameters and locals in fixed storage.

`GO TO`, written as two words, is the same as `GOTO`.

### 4.12.1 LABEL declarations

```
DECLARE (AGAIN, DONE) LABEL;
DECLARE RESTART LABEL PUBLIC;
DECLARE MONITOR LABEL EXTERNAL;
```

A label is normally declared just by labeling a statement. A LABEL declaration (PL/M-80 manual 6.3) declares it explicitly at the start of a block, before the statements (a procedure's body, a simple `DO`, or the program), and can give it an attribute.

- **Definition:** the declared label must label a statement at the **outer level of the same block**. A statement with that label inside a nested block is a different label, implicitly declared in that block.
- **Undefined labels:** a declared label that no statement has is an error if a GOTO uses it (or it is PUBLIC); otherwise it only gets a warning.
- **Factoring:** `DECLARE (A, B) LABEL PUBLIC;` declares each name, with the attribute.
- **`PUBLIC`:** the label is also exported under its own name, as `RESTART EQU G_RESTART`, so assembly code (your hooks) can jump to it.
- **`EXTERNAL`:** the label is an assembly-language label with exactly that name, for example an entry point in the hooks or a ROM monitor; `GOTO MONITOR` becomes `LBRA MONITOR`. It can't label a PL/M statement. A GOTO to it from inside a procedure resets the stack first, as for a GOTO out of a procedure.
- **Rules:**
  - no dimension, `BASED`, type or initialization;
  - PUBLIC and EXTERNAL labels only at the outer level of the main program;
  - their names can't be 6809 register names, and must not clash with other labels in the assembly (such as hook entry points).

## 4.13 Iterative DO

```
DO I = 1 TO 10;
    TOTAL = TOTAL + I;
END;

DO I = 1 TO 2 * N - 1 BY 2;     /* odd numbers */
    PROD = PROD * I;
END;
```

The index `I` must be a scalar variable, BYTE or ADDRESS (not an array element). The loop works exactly as described in the PL/M-80 manual:

1. **Start:** `start` is evaluated **once**, converted to the index's type, and assigned to the index.
2. **Test:** `limit` is evaluated and converted to the index's type. If the index is **greater than** the limit, the loop ends.
3. **Body:** the statements in the block run.

4. **Step:** `step` (1 if there's no `BY`) is evaluated and converted to the index's type, and added to the index. If the addition **wraps around** past 255 (BYTE) or 65535 (ADDRESS), the loop ends. Otherwise it goes back to step 2.

Things that follow from these rules:

- **Re-evaluation:** the limit and the step are evaluated on *every* pass, so changing a variable they use inside the body changes the loop.
- **Final value:** after a normal exit the index is the first value past the limit. `DO I = 1 TO 10` leaves `I` = 11.
- **Wrap-around:** a BYTE loop `DO I = 0 TO 255` runs 256 times and ends because the increment wraps, leaving `I` = 0. Without the wrap rule it would never end.
- **Conversions:** these can surprise you. With a BYTE index, a limit of 300 becomes 44.
- **No counting down:** arithmetic is unsigned, so `BY -1` on a BYTE index means `BY 255`, and a start above the limit runs the body zero times.
- **Step of zero:** this gives an endless loop unless the limit is passed some other way.

## 4.14 DO CASE

```
DO CASE SCORE;
    ;                               /* 0: nothing */
    CONVERSIONS = CONVERSIONS + 1;  /* 1 */
    SAFETIES = SAFETIES + 1;        /* 2 */
    DO;                             /* 3: several statements */
        FIELDGOALS = FIELDGOALS + 1;
        POINTS = POINTS + 3;
    END;
END;
```

- **Choosing a case:** the selector expression is evaluated once, and statement number *selector* runs, counting from **0**. Exactly one statement runs, then control continues after `END`.
- **Null and grouped cases:** a null statement `;` is a case that does nothing. Use a `DO; ... END;` block to make one case of several statements.
- **Out of range:** a selector of the number of cases or more is undefined in PL/M. **plmc runs no case**: it range-checks the selector and skips the block.

## 4.15 Expressions

Operators, from highest to lowest precedence:

| Precedence | Operators | Notes |
|---|---|---|
| 1 (highest) | `( )` | grouping |
| 2 | unary `-`, unary `+` | negation (two's complement) |
| 3 | `* / MOD` | multiply, divide, remainder |
| 4 | `+ - PLUS MINUS` | add, subtract; add with carry, subtract with borrow |
| 5 | `= <> < > <= >=` | comparisons |
| 6 | `NOT` | bitwise complement |
| 7 | `AND` | bitwise and |
| 8 (lowest) | `OR XOR` | bitwise or, exclusive or |

- **Grouping:** operators of equal precedence group left to right, so `A - B - C` is `(A - B) - C`. Comparisons do too: `A < B = C` compares the result of `A < B` with `C`.
- **Low precedence of `NOT`:** `NOT` binds more loosely than the comparisons, so `NOT A < B` means `NOT (A < B)`.
- **Evaluation order:** the order in which operands are evaluated is not defined, as in PL/M. Avoid expressions where a procedure call changes a variable used elsewhere in the same expression. (plmc evaluates the right operand of a binary operator before the left one; only PLUS and MINUS can tell the difference without calls, see below.)

### 4.15.1 PLUS and MINUS

```
DECLARE (AL, AH, BL, BH, SL, SH) ADDRESS;     /* 32-bit numbers AH:AL ... */
SL = AL + BL;
SH = AH PLUS BH;                              /* SH:SL = AH:AL + BH:BL */
```

`A PLUS B` is `A + B + carry`, and `A MINUS B` is `A - B - borrow` (PL/M-80 manual 12.2). They have the precedence and the result types of `+` and `-`, and are meant for arithmetic on numbers wider than 16 bits, low part first.

On the 8080 the carry is the CPU's flag, so the manual warns that the result can't be predicted. plmc defines it instead:

- **The carry is the one out of the last `+`, `-`, `PLUS` or `MINUS` executed**: the carry out of an addition, or the borrow out of a subtraction, at the operation's width (8 bits for BYTE operations, 16 for ADDRESS ones). Before any of them it is 0.
- **Nothing else changes it:** not comparisons, `*`, `/`, `MOD`, logical operators, assignments, calls, loops, or interrupt procedures (their entry code saves and restores it). The `+` that steps an iterative `DO` doesn't count either. It is kept across statements and procedure calls.
- **Within an expression,** remember that the right operand is evaluated first: in `(1 PLUS 1) PLUS (200 + 100)`, `200 + 100` sets the carry used by `1 PLUS 1`.
- **Cost:** a program that uses PLUS or MINUS keeps the carry in a byte (`RT_CARRY`) after every `+` and `-` (one `ROL` instruction each). Programs that don't use them are unaffected.

## 4.16 Types and conversions

The type of every expression is worked out at compile time:

| Expression | Result type |
|---|---|
| constant 0–255 | BYTE |
| constant 256–65535, or a two-character string | ADDRESS |
| variable, or array element | its declared type |
| structure member `X.M`, `X(i).M(j)` | the member's type |
| `LENGTH(A)`, `LAST(A)` | BYTE if 255 or less, otherwise ADDRESS |
| `SIZE(X)` | ADDRESS |
| typed procedure call | the procedure's type |
| `a + b`, `a - b`, `a PLUS b`, `a MINUS b`, `a MOD b`, `a AND b`, `a OR b`, `a XOR b` | BYTE if *both* operands are BYTE, otherwise ADDRESS |
| `a * b`, `a / b` | always ADDRESS |
| `a = b`, `a < b`, … | BYTE: `0FFH` for true, `0` for false |
| `-a`, `NOT a` | the type of `a` |

Rules that follow from this table:

- **BYTE arithmetic wraps at 8 bits, before any widening.** If `B` is a BYTE holding 200, `B + 100` is 44, *even when the result is assigned to an ADDRESS*. When you need the full result, make one operand an ADDRESS, for example by copying `B` into an ADDRESS variable first: `W = B; W = W + 100;`.
- **A constant above 255 makes the operation 16-bit.** `B + 300` is ADDRESS arithmetic.
- **Multiplication never wraps at 8 bits.** `200 * 200` is 40000.
- **Unsigned only.** All arithmetic and comparisons are unsigned. `0FFFFH > 1` is true.
- **Assignment converts:** an ADDRESS stored in a BYTE keeps its low byte, and a BYTE stored in an ADDRESS is zero-extended. The same applies to arguments and `RETURN` values.
- **Division by zero** gives a quotient of `0FFFFH` and a remainder equal to the dividend. Programs should not rely on this.

### 4.17 Conditions

`IF` and `DO WHILE` test **only bit 0** of their condition. A comparison gives `0FFH` or `0`, so this works as expected for comparisons and for `AND`, `OR` and `NOT` applied to them. For other values it means `IF 2` is false and `IF 3` is true. `AND` and `OR` always evaluate both operands; they do not short-circuit.

### 4.18 Unsupported features

These parts of PL/M-80 are recognized and rejected with an error:

| Feature | Error message |
|---|---|
| `PUBLIC` | `... is not supported` |
| embedded assignment `:=` | `embedded assignment ':=' is not supported` |
| a string of three or more characters used as a value | `strings longer than 2 characters are not supported` |

Of the PL/M-80 built-in procedures, only `LENGTH`, `LAST` and `SIZE` are provided. Others such as `HIGH`, `LOW`, `SHL`, `SHR`, `ROL` and `ROR` are not predefined. Using one gives `... is not declared`, unless you write it yourself as a procedure or an `EXTERNAL`.

## 5 Diagnostics

Errors and warnings go to standard error in the form `file:line:column: error: message`:

```
$ build/plmc e1.plm
e1.plm:3:5: error: Y is not declared
```

Compilation stops at the first error. Warnings don't stop compilation and don't change the exit status. Columns count from 1, and tabs advance to the next multiple of 8 plus 1.

| Message | Cause |
|---|---|
| `expected X, found Y` | syntax error; *X* is what the parser needed at this point |
| `X is not declared` | name used before (or without) its declaration |
| `X is already declared at line N` | second declaration in the same scope |
| `parameter X of P is not declared` | parameter without a `DECLARE` in the procedure |
| `procedure P expects N arguments, got M` | wrong number of arguments |

| Message | Cause |
|---|---|
| `X is not a procedure` | `CALL X` where X is a variable |
| `X is not a procedure or an array` | `X(...)` in an expression where X is a scalar variable |
| `X is not an array` | `X(i) = ...` where X is a scalar |
| `array A takes exactly one subscript` / `an array takes exactly one subscript` | `A(i, j)` in an expression / as a target |
| `array dimension must be from 1 to 65535, expected an array dimension (a number), found ...` | dimension 0, or not a number |
| `array X is too large (more than 65535 bytes)` | e.g. `DECLARE X(40000) ADDRESS` |
| `parameter X cannot be an array` | a parameter declared with a dimension |
| `LENGTH needs the name of a variable` (or `LAST`, `SIZE`) | argument isn't a declared variable |
| `structure X cannot be used as a value; select a member, as in X.M` | a whole structure, or a structure array's element, as a value or target |
| `X is not a structure`, `structure X has no member M` | bad member reference |
| `X is an array of structures: select one, as in X(i).M` | `X.M` without the array's subscript (outside the built-ins) |
| `X is not an array`, `X.M is not an array` | a subscript on something that isn't an array |
| `structures cannot be nested ...`, `structure members cannot be factored`, `a structure member cannot be BASED`, `member M is declared twice` | bad member list |
| `parameter X cannot be a structure` | a structure declared as a parameter |
| `a BASED variable cannot be declared AT`, `AT cannot be used with DATA`, `parameter X cannot be declared AT` | `AT` in the wrong place |
| `AT needs a number or the location of a variable` | e.g. `AT (.P)` for a procedure, or `AT (.(1, 2))` |
| `an AT address must be a number or a location reference plus or minus numbers`, `subscripts in an AT address must be constants`, `an AT address cannot use the address of a BASED variable` | e.g. `AT (2 * 100H)`, `AT (.B(I))` |
| `literal X expands recursively (more than 32 levels)` | a literal whose text uses itself |
| `a LITERALLY declaration cannot be factored, ... cannot have a dimension` | `DECLARE (A, B) LITERALLY ...`, `DECLARE A(3) LITERALLY ...` |
| `expected a string, found ...` | `LITERALLY` not followed by a string |
| `strings are limited to 255 characters` | a string over 255 characters |
| `strings longer than 2 characters are not supported` | a long string used as a value (it can be used in `INITIAL`/`DATA`, `.(...)` and `LITERALLY`) |
| `INITIAL is only allowed at the outer level of the main program (use DATA)` | `INITIAL` in a procedure or a `DO` block |
| `more initial values than scalars to initialize` | the list is longer than the declaration |
| `an initial value for a BYTE must be from 0 to 255`, `an address can only initialize an ADDRESS` | value doesn't fit its scalar |
| `an initial value must be a number, a string, or a location reference plus or minus numbers` | e.g. `INITIAL (X + 1)` or `INITIAL (2 * 3)` |

| Message | Cause |
|---|---|
| `subscripts in an initial value must be constants`, `an initial value cannot use the address of a BASED variable` | bad location reference |
| `an implicit dimension (*) needs INITIAL or DATA`, `(*) cannot be used ...` | `(*)` without a list, factored, for a structure array or member |
| `X is DATA and cannot be assigned` | assignment to a DATA variable |
| `a BASED variable cannot be initialized`, `parameter X cannot be initialized` | initialization not allowed there |
| `DATA in an --os9 module cannot hold addresses (use INITIAL)` | see OS-9 modules |
| `the index of an iterative DO must be a scalar variable` | `DO A = ...` where A is an array, a procedure, BASED, or undeclared |
| `L is not a label` | `GOTO X` where X is a variable or procedure |
| `label L is declared here but no statement at the outer level of this block has it` | a declared label used by a GOTO (or PUBLIC) but never defined |
| warning: `label L is declared but never used` | a declared label that is neither defined nor used |
| `L is declared LABEL EXTERNAL, so it cannot label a statement here` | definition of an EXTERNAL label |
| `PUBLIC and EXTERNAL labels must be declared at the outer level of the main program`, `a LABEL declaration cannot have a dimension`, `a LABEL declaration cannot be BASED` | LABEL in the wrong place or with the wrong options |
| `PUBLIC name X is also a 6809 register name`, `EXTERNAL name X is also a 6809 register name` | a label name the assembler would misread |
| `a GOTO out of a procedure must go to a label at the outer level of the main program` | the target is in an enclosing procedure, or in a block of the main program |
| `a label must be followed by a statement (use 'L: ;')` | a label right before `END` |
| `L is a label, not a variable or procedure`, `cannot assign to label L` | a label used as a value |
| `P, the base of X, is not declared` | `BASED` names an undeclared base |
| `the base of a BASED variable must be an ADDRESS scalar that is not itself BASED` | base is BYTE, an array, a procedure, or BASED |
| `parameter X cannot be BASED`, `a LITERALLY declaration cannot be BASED` | `BASED` in the wrong place |
| `the dot operator needs a variable, an array element or a procedure` | e.g. `.LENGTH(A)` |
| `the address of a procedure needs a procedure declared at the outer level, written without arguments` | `.P` for a nested procedure, or `.P(1)` |
| `a number in a constant list must fit in a byte`, `expected a number or a string, ...` | bad item in `.(...)` |
| `typed procedure F must be used in an expression, not CALLed` | `CALL` of a BYTE or ADDRESS procedure |
| `untyped procedure P cannot be used in an expression` | value taken from an untyped procedure |
| `cannot assign to procedure P` | procedure name on the left of `=` |
| `RETURN outside a procedure` | `RETURN` in the main program |
| `untyped procedure P cannot return a value` / `RETURN in typed procedure F needs a value` | `RETURN` does not match the procedure's type |

| Message | Cause |
|---|---|
| `procedure P is recursive (...) but not REENTRANT` | direct or indirect recursion through a procedure that is not REENTRANT |
| `X is not declared (only a REENTRANT procedure can use a REENTRANT procedure declared after it)` | a REENTRANT procedure used before its declaration outside a REENTRANT procedure |
| `a REENTRANT procedure must be declared at the outer level of the main program, REENTRANT procedure P cannot contain procedure declarations, an EXTERNAL procedure cannot be REENTRANT` | REENTRANT in the wrong place |
| `the variables of REENTRANT procedure P take more than 32767 bytes` | too big a stack frame |
| `INTERRUPT n: the 6809's interrupts are 0 (IRQ), 1 (FIRQ) and 2 (NMI), an INTERRUPT number must be from 0 to 255` | bad interrupt number |
| `INTERRUPT n is already used by procedure P` | two procedures for one interrupt |
| `INTERRUPT procedure P must be untyped and have no parameters, an INTERRUPT procedure must be declared at the outer level of the main program, an EXTERNAL procedure cannot be an INTERRUPT procedure` | INTERRUPT in the wrong place |
| `INTERRUPT procedures cannot be used in an --os9 module, ENABLE and DISABLE cannot be used in an --os9 module` | see OS-9 modules |
| `END X does not match ...` | label after `END` differs from the block or procedure |
| `declarations must come before statements` | `DECLARE` or procedure after the first statement |
| `declarations are only allowed in procedures and simple DO blocks` | `DECLARE` inside `DO WHILE` |
| `an EXTERNAL procedure may only declare its parameters / ... cannot have statements` | body of an `EXTERNAL` procedure |
| `EXTERNAL name X is also a 6809 register name` | external called `A`, `X`, `PC`, … |
| `invalid digit, invalid number, number ... does not fit in 16 bits` | bad numeric constant |
| `invalid character, unterminated comment, unterminated string, empty string` | lexical errors |
| warning: `typed procedure F may reach its END without RETURN` | last statement is not `RETURN` or `HALT` |
| `plmc: --org and --vectors cannot be used with --os9 ...` | OS-9 modules have no fixed address |
| `plmc: --stack is only used with --os9, --name is only used with --os9` | option without `--os9` |
| `plmc: bad --stack size ..., bad --name ...` | stack below 32 bytes, or an invalid module name |

# 6 Hooks

## 6.1 How targets are selected

The assembly plmc writes always ends with:

```
INCLUDE "plm_rt.asm"
```

```
INCLUDE "plm_hooks.asm"
```

lwasm searches its `-I` directories for these files. `plm_rt.asm` and `plm_crt0.asm` come from `runtime/`. Which `plm_hooks.asm` is used depends on which hooks directory is on the include path, so **the same `.asm` file can be assembled for different machines** without recompiling:

```
lwasm -9 -f srec -I runtime -I runtime/sim  -o prog.s19 prog.asm   # simulator
lwasm -9 -f decb -I runtime -I runtime/coco -o PROG.BIN prog.asm   # CoCo
```

## 6.2 What a hooks file must define

| Symbol | Purpose |
|---|---|
| `RT_STACKTOP` | Initial stack pointer; the stack grows down from here. The value 0 means "keep the caller's stack". |
| `PLM_EXIT` | Jumped to when the program ends, with the exit status in B (0 for a normal end). Must not return. |
| `PLM_HALT` | Jumped to by `HALT`. |
| `PLM_SETINT` | Only for programs with INTERRUPT procedures: called at start-up with B = the interrupt number (0 IRQ, 1 FIRQ, 2 NMI) and X = the entry routine; it must make that interrupt go to X. May change any register but S. |
| each `EXTERNAL` | A label with the procedure's PL/M name in upper case. |

The startup code saves the stack pointer it was entered with in `RT_ENTRYS`. A hooks file that sets `RT_STACKTOP` to 0 can therefore return to whatever started the program (BASIC, a monitor or an operating system) with `LDS RT_ENTRYS` / `RTS`. The CoCo hooks do this.

## 6.3 Calling convention for EXTERNAL procedures

- **Arguments:** each one is pushed as a **16-bit word**, from left to right. BYTE values are zero-extended first.
- **Where to find them:** on entry, the last argument is at `2,S`, just above the return address, the one before it at `4,S`, and so on. The last argument is **also in D**, so most one-argument hooks never need to touch the stack.
- **Cleanup:** the caller removes the arguments after the call (`LEAS 2n,S`).
- **Results:** a BYTE result is returned in **B** and an ADDRESS result in **D**.
- **Registers:** a hook may change A, B, X, Y, U, DP and CC. Only S must be preserved. (Y is the frame pointer of REENTRANT procedures, so in a program that has any, every call of an EXTERNAL procedure is wrapped in `PSHS Y` / `PULS Y`. The arguments are still at `2,S` and up.) Keep DP at 0 if the hook calls code that relies on it. **In `--os9` modules, U must be preserved too**, because it points at the data area, and hooks must themselves be position-independent (see OS-9 modules).
- **Runtime routines:** hooks are assembled after `plm_rt.asm`, so they may call `RT_MUL16` and `RT_DIV16`.

Example: a two-argument hook `POKE(ADDR, VALUE)`:

```
; POKE: PROCEDURE (ADDR, VALUE) EXTERNAL; DECLARE ADDR ADDRESS, VALUE BYTE;
POKE    LDX     4,S         ; first argument: ADDR
        STB     ,X          ; last argument: VALUE, also in D (B = low byte)
        RTS
```

### 6.4 Hooks provided

All four hooks files implement the same procedures, so a program that uses only these runs unchanged on any of the targets:

```
PUTCH:  PROCEDURE (C) EXTERNAL;    DECLARE C BYTE;    END PUTCH;
PUTDEC: PROCEDURE (N) EXTERNAL;    DECLARE N ADDRESS; END PUTDEC;
GETCH:  PROCEDURE BYTE EXTERNAL;                      END GETCH;
EXIT:   PROCEDURE (CODE) EXTERNAL; DECLARE CODE BYTE; END EXIT;
```

`PUTDEC` prints a number in decimal followed by a newline.

| | `runtime/sim` | `runtime/coco` | `runtime/os9` | `runtime/example` |
|---|---|---|---|---|
| `PUTCH` | simulator I/O port | Color BASIC CHROUT; LF sent as CR | `I$Write` to path 1; LF sent as CR | MC6850 ACIA, waits until ready to send |
| `GETCH` | stdin; 0 at end of input | waits for a key (POLCAT); ENTER gives 13 | `I$Read` from path 0; 0 at end of file | ACIA, waits for a character |
| `PUTDEC` | simulator decimal port | via `RT_DIV16` and CHROUT | via `RT_DIV16`, one `I$Write`, ends with CR | via `RT_DIV16`, ends with CR LF |
| `EXIT`, end of program | simulator exit status | back to BASIC (status discarded) | `F$Exit` with the status | stops in a loop |
| `HALT` | exit status 0 | back to BASIC | `F$Exit` with status 0 | stops in a loop |
| `PLM_SETINT` | stores the vector at `$FFF8`/`$FFF6`/`$FFFC` | patches BASIC's RAM vector (`JMP` at `$010C`/`$010F`/`$0109`); `PLM_EXIT` restores it | (not used) | does nothing: use `--vectors` |
| extra | | `CLS` clears the screen | | |
| stack | `$FF00` | BASIC's | in the data area (`--stack`) | `$8000` |

### 6.5 Writing hooks for a new machine

Copy `runtime/example/plm_hooks.asm` to a directory of your own, change the I/O routines and `RT_STACKTOP`, and assemble with `-I runtime -I yourdir`. Add any extra routines your programs need as further labels, and declare them in PL/M as `EXTERNAL`. The example file's `ACIA_BASE` and `RT_STACKTOP` are wrapped in `IFNDEF`, so they can also be overridden from the lwasm command line with `-D`.

## 7 OS-9 modules

With `--os9`, plmc produces an **OS-9 program module** for Microware OS-9 and NitrOS-9, Level 1 or Level 2. Like every OS-9 command, the module can be loaded at any address and shared by several processes at once.

```
build/plmc --os9 -o sum.asm sum.plm
lwasm -9 -f os9 -I runtime -I runtime/os9 -o sum sum.asm
```

`lwasm -f os9` writes the finished module, with its header parity and CRC. Copy it to the `CMDS` directory of an OS-9 disk (with toolshed: `os9 copy sum disk.dsk,CMDS/sum` and `os9 attr -e -pe disk.dsk,CMDS/sum`), then run it from the shell like any other command:

```
OS9: sum
55
OS9: sum >/d0/result
```

`sim6809 --os9 sum` runs the module without OS-9, by emulating the system calls the hooks use (see the sim6809 manual).

## 7.1 What `--os9` changes

**Position-independent code.** Calls are `LBSR` rather than `JSR`, and jumps are `LBRA` rather than `JMP`. Every other branch is already relative, so the module contains no absolute addresses.

**Data in the process's data area.** OS-9 gives each process its own data area and points U at it.

- **Variables:** declared as `RMB` offsets inside the module (which lwasm allocates in the data area, not the module), and every access is U-relative, as in `STB V_I,U`. Because the module never writes to itself, it is reentrant.
- **Order in the source:** variables come first, so lwasm knows their offsets and uses short indexed forms.
- **Hooks and stack:** the hooks' own buffers and the stack reserve follow the code.

**Startup.** `runtime/plm_crt0_os9.asm` keeps the stack, U and DP that OS-9 set up, calls the program, and ends with `F$Exit`.

**Not available:** INTERRUPT procedures, `ENABLE` and `DISABLE` (OS-9 owns the interrupts; a user program must not mask them), and `--org`/`--vectors`. Variables `AT` a number are used at that absolute address, outside the data area. REENTRANT procedures work, with their frames on the process's stack.

**Module header.**

| Field | Value |
|---|---|
| type/language | `$11` (Prgrm+Objct) |
| attributes/revision | `$81` (ReEnt, revision 1) |
| edition | 1 |
| data size | variables + hook buffers + `--stack` bytes |
| name | the PL/M program name, or `--name` |

The beginning of the `SUM` example compiled with `--os9`:

```
        PRAGMA  6809
        MOD     RT_EOM,RT_NAME,$11,$81,RT_START,RT_SIZE

; variables (data area, U-relative)
V_I     RMB     1
V_TOTAL RMB     2
```

```
RT_NAME FCS     /SUM/
        FCB     1               ; edition
        INCLUDE "plm_crt0_os9.asm"

; program SUM
PLM_MAIN
; 4: TOTAL = 0;
        LDB     #$00
        CLRA
        STD     V_TOTAL,U
...
RT_STACK RMB    256             ; stack space (--stack)
RT_SIZE EQU     .               ; data area size
        EMOD
RT_EOM  EQU     *
```

## 7.2 Module names

OS-9 finds a command by the **module name** inside the file, and expects it to match the file name in `CMDS`. By default plmc uses the PL/M program name.

- **Matching the file name:** use `--name` to give the module the name of the file you'll install, for example `--name sum`.
- **Avoiding clashes:** if a module with the same name is already in memory (or in the boot file), OS-9 runs *that* module instead. A PL/M program called `ECHO`, for instance, would run NitrOS-9's own `echo` command. Choose a program name, or `--name`, that doesn't clash.

Module names are 1 to 29 characters: a letter followed by letters, digits, `_`, `.` or `$`.

## 7.3 The OS-9 hooks

`runtime/os9/plm_hooks.asm` provides `PUTCH`, `GETCH`, `PUTDEC` and `EXIT`. They use these system calls:

| Hook | System call |
|---|---|
| `PUTCH`, `PUTDEC` | `I$Write` on path 1 (standard output) |
| `GETCH` | `I$Read` on path 0 (standard input) |
| `EXIT`, end of program, `HALT` | `F$Exit`, which gives the shell the exit status |

- **Newlines:** OS-9 ends lines with CR, so `PUTCH(10)` is sent as CR and `PUTDEC` ends its number with CR. Terminals add the line feed, and files get standard OS-9 text. Programs written for the simulator therefore print the same way under OS-9.
- **End of input:** `GETCH` returns 0 at end of file (`E$EOF`).
- **Errors:** any other I/O error ends the program with the OS-9 error code as its exit status, which the shell reports as `ERROR #nnn`.
- **Redirection:** standard input and output can be redirected as usual, for example `sum <data >/d0/out`.

**INITIAL and DATA in modules.**

- **DATA** is stored in the module, which is read-only and shared, and is read PC-relative (`V_X,PCR`). Its values can't include addresses, because those depend on where each process's data area is: use `INITIAL` for pointer tables.

- **INITIAL** variables live in the process's data area, which OS-9 creates fresh for every run. plmc stores their values in the module as a template. At start-up the program copies the template into the data area (the `RT_COPY` runtime routine) and then computes and stores any initial addresses, so they are right wherever the module and data area are.

**Rules for your own OS-9 hooks:**

- **Calls and jumps:** use only relative ones (`BSR`/`LBSR`/`BRA`/`LBRA`, or `JSR label,PCR`).
- **Data:** put it in `RMB` storage and address it through U (`LEAX buf,U`).
- **Registers:** preserve U and S.
- **System calls:** use `OS9 F$xxx`, which lwasm assembles as `SWI2` plus the function code.

# 8 Running on a Color Computer

```
build/plmc --org 3000H -o hello.asm hello.plm
lwasm -9 -f decb -I runtime -I runtime/coco -o HELLO.BIN hello.asm
```

Copy `HELLO.BIN` to a disk or disk image (for example with toolshed: `decb copy -2 -b HELLO.BIN disk.dsk,HELLO.BIN`), then in BASIC:

```
CLEAR 200,&H2FFF
LOADM"HELLO":EXEC
```

- **Load address:** always give `--org`. The default `$0100` would overwrite BASIC's RAM interrupt vectors. `CLEAR 200,&H2FFF` stops BASIC from using memory from `$3000` up.
- **Reset vector:** never use `--vectors` on a CoCo; `$FFFE` is in ROM.
- **Running and returning:** the program runs with DP = 0 on BASIC's own stack, and returns to BASIC when it ends, on `HALT`, or on `EXIT`.
- **Newlines:** Color BASIC ignores the LF character, so `PUTCH(10)` is sent as CR, the CoCo's newline. A program written for the simulator therefore prints the same way on the CoCo.
- **Lower case:** on a CoCo 1 or 2, lower-case letters appear as inverse video.
- **Interrupts:** an INTERRUPT procedure replaces BASIC's handler while the program runs, through the RAM vectors at `$0109`–`$0111`. The 60 Hz IRQ comes from PIA 0's field-sync input, which BASIC leaves enabled, and must be acknowledged by reading `$FF02` (`DECLARE PIA0B BYTE AT (0FF02H)`). BASIC's own IRQ work (`TIMER`, `PLAY`, the disk motor timeout) pauses until the program ends, when `PLM_EXIT` restores the vectors and returns to BASIC with IRQ enabled and FIRQ disabled.

The test suite checks these hooks in two ways. They run in the simulator against a stand-in for the BASIC ROM. They also run on an emulated CoCo 2 under XRoar: each program is loaded from a disk image with `LOADM` and `EXEC`, and the text screen is compared with what's expected.

# 9 Generated code

## 9.1 Layout

```
        PRAGMA  6809
        ORG     $0100                ; --org
        INCLUDE "plm_crt0.asm"       ; RT_START: set S and DP, call PLM_MAIN
PLM_MAIN                             ; main program statements
```

```
        ...
        RTS
P_PROC1                             ; each procedure
        ...
        INCLUDE "plm_rt.asm"        ; RT_MUL16, RT_DIV16
        INCLUDE "plm_hooks.asm"     ; target-specific hooks
V_X     RMB     1                   ; variables, parameters, locals
        END     RT_START
```

- **Startup:** execution starts at `RT_START`, at the `--org` address. It sets the stack (unless `RT_STACKTOP` is 0) and DP = 0, calls `PLM_MAIN`, and jumps to `PLM_EXIT` with status 0.
- **Variables** come last and are reserved with `RMB`, so they add nothing to S-record or `.BIN` files.
- **Total size:** for a typical small program, the code is a few hundred bytes and the runtime about 70 bytes.

With `--os9` the layout is that of an OS-9 module instead; see What `--os9` changes.

## 9.2 Example

The program from Building and quick start compiles to (in part):

```
; 6: DO WHILE I <= 10;
L1
        LDB     V_I
        CMPB    #$0A
        LBHI    L2
; 7: TOTAL = TOTAL + I;
        LDB     V_I
        CLRA
        PSHS    D
        LDD     V_TOTAL
        ADDD    ,S++
        STD     V_TOTAL
; 8: I = I + 1;
        LDB     V_I
        ADDB    #$01
        STB     V_I
        LBRA    L1
L2
```

Each statement is preceded by its source line as a comment, which makes the listing and simulator traces easy to follow.

## 9.3 Conventions

- **Registers:** a BYTE value is computed in B and an ADDRESS value in D. A BYTE is widened with `CLRA` only where a 16-bit value is needed.
- **Binary operators:** the right operand is evaluated first and pushed, then the left one, then they're combined with `op ,S+` or `op ,S++`. When the right operand is a constant or a variable of the right size, it's used directly as the instruction operand (`ADDB V_I`, `CMPB #$0A`).

- **Comparisons:** these use the unsigned branches `BLO`, `BHS`, `BHI`, `BLS`, `BEQ` and `BNE`.
  - In an `IF` or `WHILE`, the code branches straight on the comparison.
  - As a value, a comparison produces `$FF` or `$00` in B.
  - Statement-level control flow uses long branches, so there's no limit on the size of a block.
- **Iterative DO:**
  - The start value is stored in the index.
  - At the top of the loop the index is compared with the limit, which is re-evaluated there (`LBHI`/`LBLO` to the exit).
  - After the body the step is added with `ADDB`/`ADDD`, and the result stored. `LBCC` goes back to the top; a carry means the addition wrapped, and the loop ends.
- **DO CASE:**
  - The selector is range-checked (`CMPD #n` / `LBHS` past the block), then doubled.
  - `LEAX table,PCR` / `LDD D,X` / `JMP D,X` jumps through a table of `FDB case-table` *offsets*. Offsets rather than addresses keep the code position-independent, so the same sequence works in `--os9` modules.
  - Each case ends with `LBRA` past the block.
- **GOTO:**
  - Every statement leaves the stack as it found it, so a GOTO within a procedure, or within the main program, is just `LBRA`.
  - If any GOTO leaves a procedure, `PLM_MAIN` saves its stack pointer in `RT_MAINS` on entry (`STS RT_MAINS`). Such a GOTO is `LDS RT_MAINS` / `LBRA label`, which discards the return addresses and intermediate values of the calls it leaves.
- **BASED variables:**
  - The base is loaded into X (`LDX P`, or `LDX P,U`), and the variable is used as `,X` or `k,X`, so a store needs no extra registers.
  - Computed subscripts use the same `D,X` sequence as ordinary arrays, with the base's value as X.
- **Dot operator:**
  - Normally `LDD #V_X`. In `--os9` modules it is `LEAX V_X,U` / `TFR X,D` for a variable, and `LEAX P_X,PCR` / `TFR X,D` for a procedure, so the code stays position-independent.
  - Constant lists become `FCB` data after the procedures.
- **INITIAL and DATA:**
  - In a normal program, INITIAL variables are emitted as `FCB`/`FDB` in the data section, so the values are part of the load image. The rest of a partly initialized variable is `RMB`.
  - DATA variables are emitted the same way, among the code.
  - Addresses become label expressions such as `FDB V_TABLE+2`.
  - In `--os9` modules, see OS-9 modules.
- **AT:** a variable declared `AT` gets no storage: its label is an `EQU`, `M_X EQU $FF00` for a fixed address or `V_X EQU V_BUFFER+1` for an alias. `M_X` is used as a plain absolute operand even in `--os9` modules.
- **Structures:**
  - A member reference whose subscripts are all constants is a fixed location, `V_X+offset` (structure subscript × structure size + member offset + member subscript × member size), and is used directly as an operand.

  - Computed subscripts are scaled with shifts when the size is a power of two, and with `RT_MUL16` otherwise. Two computed subscripts are added on the stack.
  - BASED structures and the dot operator use the same paths as arrays.

- **Array elements:**
  - *Constant subscript:* the element is an ordinary operand, `V_A+6` (or `V_A+6,U` in an OS-9 module).
  - *Computed subscript:* the subscript is computed into D and doubled for ADDRESS arrays. The array's address goes into X (`LDX #V_A`, or `LEAX V_A,U`), and the element is loaded with `LDB D,X` / `LDD D,X`.
  - *Storing to a computed element:* the value is pushed first, then the address is formed with `LEAX D,X`, and the value is pulled and stored through X.

- **PLUS and MINUS:** when a program uses them, every `+`, `-`, `PLUS` and `MINUS` is followed by `ROL RT_CARRY`, which shifts the carry into bit 0 of that byte without touching A or B. PLUS is `LSR RT_CARRY` (the carry back into C) and `ADCB`, plus `ADCA` on the high bytes for ADDRESS (the 6809 has no 16-bit add with carry); MINUS uses `SBCB`/`SBCA`.

- **Multiplication and division:** BYTE × BYTE uses the 6809's `MUL`. All other multiplications and all divisions call `RT_MUL16` or `RT_DIV16` (left operand in X, right in D).

- **REENTRANT procedures:**
  - The caller pushes the arguments, first to last, each in its declared width (BYTE 1 byte, ADDRESS 2), calls the procedure, and removes them with `LEAS n,S`.
  - The procedure starts with `PSHS Y`, `LEAS -n,S` for its variables and `TFR S,Y`, and returns with `LEAS n,Y` and `PULS Y,PC` (B and D, the result, are untouched).
  - Its parameters and variables are then at fixed offsets from Y, emitted as `EQU`s before the procedure (`V2_N EQU 4`) and used as `V2_N,Y`:

| Offset from Y | Contents |
|---|---|
| 0 and up | variables, in declaration order |
| n | the caller's Y |
| n+2 | return address |
| n+4 and up | arguments, the last one first |

  - Y belongs to the running REENTRANT procedure. Ordinary procedures and the runtime don't use it, REENTRANT procedures restore it, and EXTERNAL calls save it.

- **INTERRUPT procedures:** `PLM_MAIN` starts with `LDX #RT_INTn` / `LDB #n` / `JSR PLM_SETINT` for each one. The entry routine `RT_INTn` follows the procedures: `CLRA`, `TFR A,DP`, `JSR P_X`, `RTI`, with `PSHS U,Y,X,DP,D` / `PULS D,DP,X,Y,U` around it for FIRQ. `ENABLE` is `ANDCC #$AF`, `DISABLE` is `ORCC #$50`, and a `CALL` of an interrupt procedure is followed by `ANDCC #$AF`.

- **Procedure calls:** arguments are stored in the procedure's fixed parameter locations and the procedure is called with `JSR`. If an argument itself contains a call, all arguments are evaluated and pushed first, then stored. That makes `F(1, F(2, 3))` work even though `F` has only one set of parameter locations.

- **Label names:** PL/M names can't contain `_`, so these labels never clash with yours:

| Label | Meaning |
|---|---|
| `V_X` | variable `X` of the program |
| `Vn_X` | variable or parameter `X` of nested scope $n$ |
| `M_X`, `Mn_X` | variable `X` declared `AT` a fixed address |

| Label | Meaning |
|---|---|
| `P_X`, `Pn_X` | procedure `X` |
| `G_X`, `Gn_X` | statement label `X` (in scope $n$) |
| `Lnnn` | compiler-generated label |
| `RT_*` | runtime routine or data |
| `PLM_*` | entry point the hooks provide |
| `X` | `EXTERNAL` procedure `X` |

# 10 Inspecting the compiler's view

`--tokens` prints each token with its line and column. Numbers are shown with their value, which is useful for checking how a constant was read. Strings are printed as `STRING 'text'`, with the value of one- and two-character strings. `--tokens` shows the source as written; LITERALLY names are expanded later, by the parser.

```
2:1 IDENT X
2:3 PUNCT =
2:5 NUMBER 255
2:10 PUNCT +
```

`--ast` prints the syntax tree after parsing, and after LITERALLY substitution, with expressions written as prefix lists. It shows exactly how precedence was applied:

```
PROGRAM SUM
  PROCEDURE PUTDEC (N) EXTERNAL
    DECLARE N ADDRESS
  DECLARE I BYTE
  DECLARE TOTAL ADDRESS
  ASSIGN TOTAL = 0
  ASSIGN I = 1
  DO WHILE (<= I 10)
    ASSIGN TOTAL = (+ TOTAL I)
    ASSIGN I = (+ I 1)
  CALL (CALL PUTDEC TOTAL)
```

To follow a program at run time, assemble with a listing (`lwasm ... --list=prog.lst`) and run `sim6809 -t`. The trace addresses match the listing, and the listing shows the PL/M source line for each group of instructions.

# 11 Testing

`make test` runs all of these suites:

| Suite | What it checks |
|---|---|
| `lex`, `parse` | `--tokens` and `--ast` output for every number form, every keyword, precedence, dangling `ELSE` and nested procedures, against golden files |

| Suite | What it checks |
|---|---|
| `err` | 177 programs that must fail, each with the expected message and position |
| `run` | complete programs compiled, assembled and run in `sim6809`; output and exit status compared. They cover every operator and conversion rule, control flow including iterative DO (wrap-around, re-evaluated limit and step, conversions) and DO CASE, input, procedures (including nested calls), arrays (a sieve, a sort, edge cases), structures (the manual's AIRPLANE, PAYROLL, POINT and 100×100 examples, built-ins on every reference form, BASED structures, a linked list), BASED variables and the dot operator (the manual's examples, BASED arrays and parameters, constant lists), INITIAL and DATA (the manual's examples, `(*)`, strings in BYTE and ADDRESS arrays, pointer tables, DATA in a procedure), PLUS and MINUS (32- and 24-bit arithmetic, and under interrupts), labels and GOTO (out of loops, blocks and nested procedure calls; LABEL declarations, PUBLIC and EXTERNAL labels; 10000 jumps out of three calls to check the stack is reset), AT (aliases, and variables at sim6809's I/O ports), REENTRANT procedures (the manual's SUM$ARRAY$, $indirectrecursion$, $localarraysandstructures$, $Ackerm$ $org`and` -$ $-vectors`||`fuzz`|randomprograms$, $whosearraysandstructuresares$ $bytestructures$, $whosesubscriptsneedamultiply$; $carrystatementswith$ $-PLUSMINUS`$, $modelledintheorderplmcevaluatesthem$; $halfofth$ $testN =$ $2000`runsmore||`coco`,`xroar`|theColorComputerhooks$, $inthesimul$ $syncIRQs)(the`xroar`suiteisskippedifXRoar$, $theROMsortoolshed$ $-os9`andrunby`sim6809 -$ $-os9`(programsthatneedcompileroptions$, $suchasfixedaddresses$, $ar$ $9Level2disk$, $bootedonanemulatedCoCo3underXRoar$, $witheachprog$ `bootable NitrOS-9 Level 2 disk image)` `and$COCO3_ROM; skipped if they, XRoar or toolshed are missing \| \|sim‘` |

## 12 Source code

| File | Contents |
|---|---|
| `src/main.c` | command line, file handling |
| `src/lexer.c` | tokens, numbers, strings, comments |
| `src/parser.c`, `ast.c` | recursive-descent parser, syntax tree, `--ast` |
| `src/symtab.c`, `sema.c` | scopes, name resolution, types, call checks, recursion detection |
| `src/codegen.c` | assembly output |
| `runtime/plm_crt0.asm`, `plm_crt0_os9.asm`, `plm_rt.asm` | startup code (normal and OS-9), 16-bit multiply and divide |

| File | Contents |
|---|---|
| `runtime/*/plm_hooks.asm` | hooks for the simulator, the CoCo, OS-9, and the ACIA template |